

# CSE563 — Concept Sheet, Block C

## Unit IV: Graph Theory

Theory reference — every concept, no numericals

School of Computer Science & Engineering

Session 2025–26



### What This Sheet Is

A **theory-only** reference for Unit IV, taught as a standalone block because graph theory is self-contained — it needs only the relations and functions of Unit I, and it feeds forward into the algorithm design of Units V and VI. Every definition, theorem, algorithm description and comparison that can be examined in written form appears here. Traces and calculations live in the *Unit IV Numerical Guide*; practice questions live in the *Unit IV Worksheet*.

## Contents

# 1 Graph Basics

## Core Definitions

**Graph** —  $G = (V, E)$  — a set of *vertices* and a set of *edges* joining pairs of them. Write  $n = |V|$ ,  $m = |E|$ .

**Undirected vs directed** — an undirected edge is an unordered pair  $\{u, v\}$ ; a directed edge (*arc*) is an ordered pair  $(u, v)$ .

**Adjacent / incident** — two vertices sharing an edge are adjacent; an edge is incident to its endpoints.

**Loop** — an edge from a vertex to itself. **Parallel edges**: two or more edges on the same pair.

**Simple graph** — no loops and no parallel edges.

**Degree** —  $\deg(v)$  = number of incident edges (a loop counts twice). In a digraph, *in-degree* and *out-degree*.

**Isolated / pendant vertex** — degree 0 / degree 1.

**Subgraph** —  $V' \subseteq V$ ,  $E' \subseteq E$ . A *spanning* subgraph uses every vertex.

## Walks, Paths and Connectivity

| Term    | Definition                                   | Repeats allowed?            |
|---------|----------------------------------------------|-----------------------------|
| Walk    | any sequence of consecutively incident edges | vertices and edges          |
| Trail   | a walk with no repeated edge                 | vertices only               |
| Path    | a walk with no repeated vertex               | none                        |
| Circuit | a closed trail                               | vertices only               |
| Cycle   | a closed path                                | none (except the endpoints) |

**Length** — the number of edges in the walk.

**Connected** — an undirected graph in which a path exists between every pair of vertices. The maximal connected pieces are **connected components**.

**Strongly connected** — a digraph in which every vertex reaches every other by *directed* paths.

**Cut vertex (articulation point)** — a vertex whose removal disconnects the graph. **Bridge**: an edge whose removal disconnects.

## The Handshaking Lemma

$$\sum_{v \in V} \deg(v) = 2|E|.$$

**Why**: every edge contributes exactly 2 to the total degree count, one at each endpoint.

**Corollary**. The number of odd-degree vertices is always **even**. This is the standard tool for showing that a proposed degree sequence is impossible.

**In a digraph**:  $\sum \deg^-(v) = \sum \deg^+(v) = |E|$ .

## Representations

|                  | Space           | Edge lookup | List neighbours of $u$ |
|------------------|-----------------|-------------|------------------------|
| Adjacency matrix | $\Theta(n^2)$   | $\Theta(1)$ | $\Theta(n)$            |
| Adjacency list   | $\Theta(n + m)$ | $O(\deg u)$ | $\Theta(\deg u)$       |

**Choice** — matrices suit *dense* graphs and lookup-heavy code; lists suit *sparse* graphs, which is the usual real case. BFS and DFS achieve  $\Theta(n + m)$  only with adjacency lists.

**Walk counting** —  $(A^k)_{ij}$  gives the number of walks (repeats permitted) of length exactly  $k$  from  $i$  to  $j$ .

**Incidence matrix** —  $n \times m$ , rows indexed by vertices and columns by edges.

## 2 Graph Types

### The Standard Families

| Type                    | Definition and key facts                                                                                             | Where it appears                  |
|-------------------------|----------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Complete $K_n$          | every pair adjacent; $\binom{n}{2}$ edges; every degree $n - 1$                                                      | fully-meshed networks             |
| Regular ( $k$ -regular) | every vertex has degree $k$ ; $K_n$ is $(n - 1)$ -regular                                                            | peer-to-peer overlays             |
| Bipartite               | $V = X \cup Y$ with every edge crossing. Bipartite $\iff$ no odd cycle $\iff$ 2-colourable. $K_{m,n}$ has $mn$ edges | users-items, jobs-machines        |
| Tree                    | connected and acyclic                                                                                                | file systems, parse trees         |
| Forest                  | a disjoint union of trees                                                                                            | —                                 |
| DAG                     | directed and acyclic; always admits a topological order                                                              | build dependencies, prerequisites |
| Planar                  | drawable without crossings                                                                                           | PCB and VLSI layout               |
| Weighted                | a number on each edge (cost, distance, capacity)                                                                     | road maps, networks               |

### Trees — Equivalent Characterisations

For a graph  $T$  on  $n$  vertices, the following are equivalent:

1.  $T$  is connected and acyclic (a tree);
2.  $T$  is connected with exactly  $n - 1$  edges;
3.  $T$  is acyclic with exactly  $n - 1$  edges;
4. any two vertices are joined by exactly *one* path;
5.  $T$  is minimally connected — every edge is a bridge.

**Consequences** — adding any edge to a tree creates exactly one cycle; removing any edge disconnects it. Every finite tree with  $n \geq 2$  has at least two leaves.

**Spanning tree** — a spanning subgraph that is a tree; exists iff the graph is connected.

**Rooted tree** — one vertex distinguished as the root, giving parent, child, depth and height. A binary tree of height  $h$  holds at most  $2^{h+1} - 1$  nodes.

### Planarity

**Euler's formula** — for a connected planar graph drawn without crossings,  $n - m + f = 2$ , where  $f$  counts the faces (including the outer one).

**Edge bounds** —  $m \leq 3n - 6$  for a simple planar graph with  $n \geq 3$ ; and  $m \leq 2n - 4$  for a *bipartite* planar graph, since every face needs at least four edges.

**Kuratowski's obstructions** —  $K_5$  and  $K_{3,3}$  are the two minimal non-planar graphs.  $K_5$  fails  $m \leq 3n - 6$  ( $10 > 9$ );  $K_{3,3}$  fails the bipartite bound ( $9 > 8$ ).

### Eulerian and Hamiltonian — the Instructive Contrast

|                      | Eulerian (every <i>edge</i> once)                    | Hamiltonian (every <i>vertex</i> once) |
|----------------------|------------------------------------------------------|----------------------------------------|
| Circuit condition    | connected and <i>all</i> degrees even                | no known simple characterisation       |
| Path condition       | connected and <i>exactly two</i> odd-degree vertices | —                                      |
| Deciding it          | $O(n + m)$ — just check degrees                      | <b>NP-complete</b> (Unit V)            |
| Sufficient condition | —                                                    | Dirac: $\deg(v) \geq n/2$ for all $v$  |
| Application          | route inspection, DNA assembly                       | travelling salesman, scheduling        |

**Why the asymmetry matters.** Eulerian is a *local* property — it depends only on individual

degrees, so it is checkable instantly. Hamiltonicity is *global*, depending on how the whole graph fits together, and no polynomial test is known (nor can one exist unless  $P = NP$ ). This contrast is the clearest preview of Unit V's complexity theory.

### 3 Graph Traversal

#### Breadth-First Search

**Mechanism** — a **queue** (FIFO). Explore in layers: everything at distance 1, then 2, and so on.

**Complexity** —  $\Theta(n + m)$  with adjacency lists.

**Central property** — in an *unweighted* graph, BFS finds a **shortest path** (fewest edges) from the source to every reachable vertex. The tree of first-discovery edges is the BFS tree, and each tree path is a shortest path.

**Uses** — shortest unweighted paths, connected components, bipartiteness testing (colour by level parity), web crawling by depth, and the shortest-augmenting-path rule that turns Ford–Fulkerson into Edmonds–Karp.

**Memory** — proportional to the *width* of the graph, which can be very large on broad graphs.

#### Depth-First Search

**Mechanism** — a **stack**, or equivalently recursion. Go as deep as possible, then backtrack.

**Complexity** —  $\Theta(n + m)$ .

**Edge classification** — *tree* edges (first discovery), *back* edges (to an ancestor — each proves a **cycle**), *forward* edges, and *cross* edges.

**Uses** — cycle detection, topological sorting, strongly connected components (Kosaraju, Tarjan), articulation points and bridges, maze solving, and backtracking search (Unit VI).

**Memory** — proportional to the *depth*, usually far smaller than BFS's width.

**Warning** — DFS does **not** find shortest paths — it may reach a nearby vertex by a very long route.

#### Topological Sort

**Definition** — a linear ordering of a DAG's vertices such that every directed edge points forward in the order. It exists *iff* the graph is acyclic.

**Kahn's algorithm** — repeatedly output a vertex of in-degree 0 and remove it, decrementing the in-degrees of its successors. If vertices remain but none has in-degree 0, a cycle exists.

**DFS method** — output vertices in order of *decreasing finish time*.

**Non-uniqueness** — most DAGs admit several valid orders; any ordering satisfying every edge is correct.

**Uses** — build systems (**make**), course prerequisites, task scheduling, spreadsheet recalculation, instruction scheduling.

### 4 Shortest Paths and Spanning Trees

#### Dijkstra's Algorithm

**Mechanism** — maintain a tentative distance  $d[v]$ ; repeatedly extract the unvisited vertex of smallest  $d$ , declare it *final*, and **relax** each outgoing edge — update  $d[v]$  if  $d[u] + w(u, v)$  is strictly smaller.

**Complexity** —  $O((n + m) \log n)$  with a binary heap;  $O(m + n \log n)$  with a Fibonacci heap.

**Requirement** — **non-negative weights only**. The greedy step — finalising a vertex on

extraction — assumes that extending a path cannot reduce its cost. A single negative edge breaks this and produces wrong answers.

**Key habit** — a vertex's label may improve *after* it first becomes finite; relaxation continues until the vertex is extracted.

### Choosing a Shortest-Path Algorithm

| Situation                            | Algorithm      | Complexity                                    |
|--------------------------------------|----------------|-----------------------------------------------|
| Unweighted                           | BFS            | $\Theta(n + m)$                               |
| Non-negative weights, one source     | Dijkstra       | $O((n + m) \log n)$                           |
| Negative weights (no negative cycle) | Bellman–Ford   | $O(nm)$ ; also <i>detects</i> negative cycles |
| All pairs                            | Floyd–Warshall | $\Theta(n^3)$ — dynamic programming (Unit VI) |
| With a heuristic (maps, games)       | A*             | Dijkstra ordered by $d[v] + h(v)$             |

### Minimum Spanning Trees

**Problem** — in a connected weighted graph, find the spanning tree of minimum total edge weight.

**Kruskal** — sort all edges by weight; accept each unless it closes a cycle (tested with union–find).  $O(m \log m)$ .

**Prim** — grow a single tree from an arbitrary start vertex, always adding the cheapest edge leaving the tree.  $O(m \log n)$  with a heap — structurally like Dijkstra, but keyed on the *edge weight* rather than the accumulated path length.

**Cut property** — the lightest edge crossing any cut belongs to some MST. This single fact proves both algorithms correct.

**Uniqueness** — if all edge weights are distinct, the MST is unique. With ties, several MSTs of equal weight may exist — but the total weight is always the same.

### MST versus Shortest-Path Tree — a Standard Examination Trap

They optimise different quantities and are generally *different trees*:

- The **MST** minimises the *sum of all edge weights*. It has no root and makes no promise about any individual path.
- The **shortest-path tree** minimises the *distance from a fixed root* to each vertex separately, and may use heavier edges to do so.

A triangle with sides 2, 2, 3 separates them: the MST takes the two 2-edges (total 4), while Dijkstra from the vertex opposite the 3-edge takes one 2-edge and the 3-edge (total 5), because routing through the far vertex would cost 4 rather than 3.

## 5 Graph Colouring

### Definitions and Bounds

**Proper vertex colouring** — an assignment of colours such that no two adjacent vertices share one.

**Chromatic number**  $\chi(G)$  — the minimum number of colours in a proper colouring.

**Clique number**  $\omega(G)$  — the size of the largest complete subgraph.

**Maximum degree**  $\Delta(G)$  — the largest vertex degree.

$$\omega(G) \leq \chi(G) \leq \Delta(G) + 1.$$

**Lower bound:** a clique of size  $\omega$  needs  $\omega$  distinct colours among its own vertices. **Upper bound:** greedy colouring in any order always finds a free colour, since at most  $\Delta$  neighbours are already coloured.

**Brooks' theorem** —  $\chi \leq \Delta$  unless  $G$  is a complete graph or an odd cycle.

### Standard Values and the Four Colour Theorem

| Graph                           | $\chi$   | Reason                                  |
|---------------------------------|----------|-----------------------------------------|
| Edgeless                        | 1        | nothing conflicts                       |
| Tree ( $n \geq 2$ ), even cycle | 2        | bipartite — no odd cycle                |
| Odd cycle $C_{2k+1}$            | 3        | two colours fail where the cycle closes |
| Complete $K_n$                  | $n$      | every pair adjacent                     |
| Any planar graph                | $\leq 4$ | <b>Four Colour Theorem</b>              |

**Bipartiteness test** —  $\chi(G) = 2$  iff  $G$  has an edge and contains no odd cycle. Checked in  $\Theta(n + m)$  by BFS, colouring by level parity and failing if any edge joins two vertices of equal parity.

**Complexity** — deciding whether  $\chi(G) \leq 3$  is **NP-complete** (Unit V), which is why practical tools use heuristics.

### Welsh–Powell (Greedy) Colouring

1. Order the vertices by *decreasing degree*.
2. Give the first uncoloured vertex a new colour, and give that same colour to every later uncoloured vertex not adjacent to any vertex already holding it.
3. Repeat with the next colour until every vertex is coloured.

**Guarantee** — uses at most  $\Delta + 1$  colours, and is fast — but it is a *heuristic* and need not achieve  $\chi(G)$ .

**Proving minimality** — a colouring with  $k$  colours is only proven optimal by exhibiting a clique of size  $k$  (so that  $\chi \geq k$ ). Stating “three colours were enough” does not establish that two would not be.

### Applications of Colouring

**Examination / lecture timetabling** — vertices are exams, edges are shared students, colours are time slots.

**Register allocation** — vertices are program variables, edges join variables live at the same time, colours are CPU registers. When  $\chi$  exceeds the register count the compiler must *spill* a variable to memory.

**Frequency assignment** — transmitters that would interfere must receive different channels.

**Sudoku** — a 9-colouring problem on an 81-vertex graph.

**Edge colouring** — adjacent *edges* get different colours; models round-robin tournaments. Vizing:  $\Delta \leq \chi'(G) \leq \Delta + 1$ .

## 6 Network Flow

### Definitions

**Flow network** — a digraph with a source  $s$ , a sink  $t$ , and a non-negative capacity  $c(u, v)$  on each arc.

**Flow** — an assignment  $f(u, v)$  satisfying two constraints — the **capacity constraint**  $0 \leq f(u, v) \leq c(u, v)$ , and **flow conservation** (in = out) at every vertex other than  $s$  and  $t$ .

**Value**  $|f|$  — the net flow leaving the source, equivalently the net flow entering the sink.

**$s$ - $t$  cut** — a partition  $(S, T)$  of the vertices with  $s \in S$  and  $t \in T$ .

**Cut capacity** — the sum of the capacities of arcs directed **from**  $S$  **to**  $T$  **only**. Arcs running from  $T$  back into  $S$ , and arcs with both endpoints on the same side, are *not* counted.

### The Residual Network — and Why Reverse Arcs Exist

For each arc  $u \rightarrow v$  the residual network carries:

- a **forward** arc  $u \rightarrow v$  of capacity  $c(u, v) - f(u, v)$ , the spare capacity;
- a **reverse** arc  $v \rightarrow u$  of capacity  $f(u, v)$ , representing flow that may be *cancelled*.

**Why the reverse arc is essential.** A greedy augmenting path can commit flow to an arc that a better solution needs elsewhere. The reverse arc lets a later augmentation retract that decision and reroute. Without it the method would be merely greedy and would stop short of the maximum; with it, Ford–Fulkerson is provably optimal. *Omitting reverse arcs is the single most common error in this topic.*

### Ford–Fulkerson and Max-Flow Min-Cut

**Method.** Start from zero flow. While an  $s \rightarrow t$  path exists in the residual network, push its bottleneck (minimum residual capacity) along it and update the residuals. Stop when no augmenting path remains.

**Max-Flow Min-Cut Theorem.**

$$\max_f |f| = \min_{(S,T)} c(S, T).$$

The following three statements are equivalent: (i)  $f$  is a maximum flow; (ii) the residual network contains no augmenting path; (iii)  $|f| = c(S, T)$  for some cut.

**Reading off the minimum cut** — after termination,  $S$  is the set of vertices *reachable from  $s$  in the final residual network* (reverse arcs included);  $T$  is everything else. Every  $S \rightarrow T$  arc will be saturated and every  $T \rightarrow S$  arc empty.

**Complexity** — with BFS choosing the shortest augmenting path (**Edmonds–Karp**) the bound is  $O(nm^2)$ . With arbitrary path choice and irrational capacities the method may fail to terminate.

**Integrality** — with integer capacities there is always an integral maximum flow — the fact that makes the matching reduction work.

### Applications of Flow

**Maximum bipartite matching** — join  $s$  to every left vertex and every right vertex to  $t$ , all capacities 1; the maximum flow equals the maximum matching. *Hall’s theorem* says a perfect matching exists iff every set  $W$  of left vertices has  $|N(W)| \geq |W|$ .

**Network reliability** — the minimum cut identifies the links whose failure costs the most throughput — the true bottleneck.

**Image segmentation** — pixels are vertices,  $s$  is the foreground and  $t$  the background; the minimum cut traces the object boundary.

**Others** — project selection, airline scheduling, baseball elimination — all reduce to max flow.

💡 **Exam Tip & Key takeaway: Unit IV — What the Examiner Is Actually Checking**

- **The table, not the answer.** Every algorithm question expects a step-by-step trace. A bare final answer earns little credit even when correct.
- **Stated tie-breaks.** Where several vertices or edges qualify, say which rule you applied.
- **Reverse arcs** in every residual computation, and **only  $S \rightarrow T$  arcs** in every cut capacity.
- **Minimality arguments** for colouring must exhibit a clique, not merely report the number of colours used.
- **Verification.** Max flow = cut capacity; MST has  $n - 1$  edges; BFS distance = tree-path length; degree sum =  $2m$ . Each is one line and each catches real errors.
- **The forward link.** Eulerian is easy and Hamiltonian is NP-complete; 2-colouring is easy and 3-colouring is NP-complete. These pairs are the bridge into Unit V.